

2026-05-05-p1-f04-git-worktree-agent-isolation-design

P1-F04 — Git Worktree Agent Isolation — Design Spec

Date: 2026-05-05 **Author:** Claude Opus 4.7 (1M context) + user (milos85vasic.2nd@gmail.com)
Phase / Feature: Phase 1, Feature 4 of docs/superpowers/specs/2026-05-04-cli-agent-fusion-synthesis-design.md **Status:** APPROVED in brainstorming, awaiting user review of written spec **Successor:** to be handed to superpowers:writing-plans for executable plan **Predecessor:** Feature 3 (Tool Result Persistence) — closed 2026-05-05 (commits f813fc9...0b92630)

1. Goals, non-goals, success criteria

1.1 What we're building

A claude-code-style git-worktree isolation feature for HelixCode. Agents (and humans) can enter a named, validated worktree (`/[a-zA-Z0-9._-]+/`, max 64 chars) at `<repoRoot>/.helix-worktrees/<name>/`, work in parallel branches without polluting main, and exit back to the main worktree when done. The feature exposes:

- 4 agent tools (EnterWorktree, ExitWorktree, ListWorktrees, RemoveWorktree) registered with `internal/tools/registry.ToolRegistry`.
- 4 Cobra subcommands `helixcode worktree {list,enter,exit,remove}`. The stateful enter/exit print a help message and exit non-zero when run from outside an interactive `helixcode chat` session.
- 1 `/worktree` slash command (`/worktree [list|enter <name> [branch]|exit|remove <name>]`) registered with `internal/commands/builtin`.

The implementation shells out to the `git` binary for all worktree operations, consistent with the existing `internal/tools/git/` package. It does NOT initialise submodules — agents that need submodule code run `git submodule update --init --recursive` from inside the worktree.

1.2 Goals (priority order)

- **G1 — No bluff.** Per Constitution Article XI §11.9, every PASS in this feature carries positive runtime evidence. The Challenge proves a real `git worktree add` succeeded against an ephemeral repo and the agent's effective working directory actually changed.
- **G2 — Extend, don't parallelise.** Same lesson as F01/F02/F03. New sub-package `internal/tools/worktree/` lives next to `permissions/`, `persistence/`, `confirmation/` — same logical layer, different concern. The existing `internal/tools/git/` package (focused on auto-commit + attribution + message generation) is

left untouched: worktree management is a workflow concern, not a commit-metadata concern.

- **G3 — Per-session state via single field on `session.Manager`.** The porting doc proposes a parallel `internal/session/worktree_state.go`; we instead add a single `currentWorktree` string field plus getter/setter to `internal/session/manager.go`. Less surface area, same outcome.
- **G4 — Meta-only worktrees.** `EnterWorktree` does not init submodules. The new worktree contains the meta-repo files + the inner Go module at `helix_code/` (a tracked subdirectory, present in any checkout). Submodules under `helix_agent/`, `dependencies/`, etc. are empty placeholder directories. Agents that need submodule code run `git submodule update --init --recursive` from inside the worktree.
- **G5 — Full surface (tools + Cobra + slash).** Mirrors F02's polish level. Agent-facing tools, human-facing CLI, and in-session slash command share a single `*Manager` instance.

1.3 Non-goals (explicit out-of-scope for F04)

- **N1.** Subagent isolation (`isolation: "worktree"` on Task dispatch). Belongs to P1-F15 (Subagent Team).
- **N2.** Cross-session worktree resume (auto-restore "I was in worktree X"). Belongs to P1-F11 (Session Transcript Resume).
- **N3.** Submodule auto-init. Out of scope per Q2 (option A).
- **N4.** Worktree-aware Read/Write/Bash tools. F04 adds `Manager.GetCurrentDirectory()` and `Manager.IsIsolated()` so future tasks can wire tools to honour the active worktree; F04 itself does NOT modify existing tool implementations. Until that wiring lands (a Phase 3 follow-up or a later P1 feature), the agent must use absolute paths returned by `EnterWorktree`.
- **N5.** F02/F03 follow-ups (permissions engine threading into `ConfirmationCoordinator`; persistence dispatcher wiring). Tracked separately in PROGRESS.md parking lot.
- **N6.** `go-git` library. F04 shells out to the `git` binary, consistent with `internal/tools/git/`.
- **N7.** Worktrees outside the repo (`<repoRoot>/.../...` or `~/helixcode/worktrees/...`). Q3 ratified the in-repo location `<repoRoot>/helix-worktrees/<name>/`.

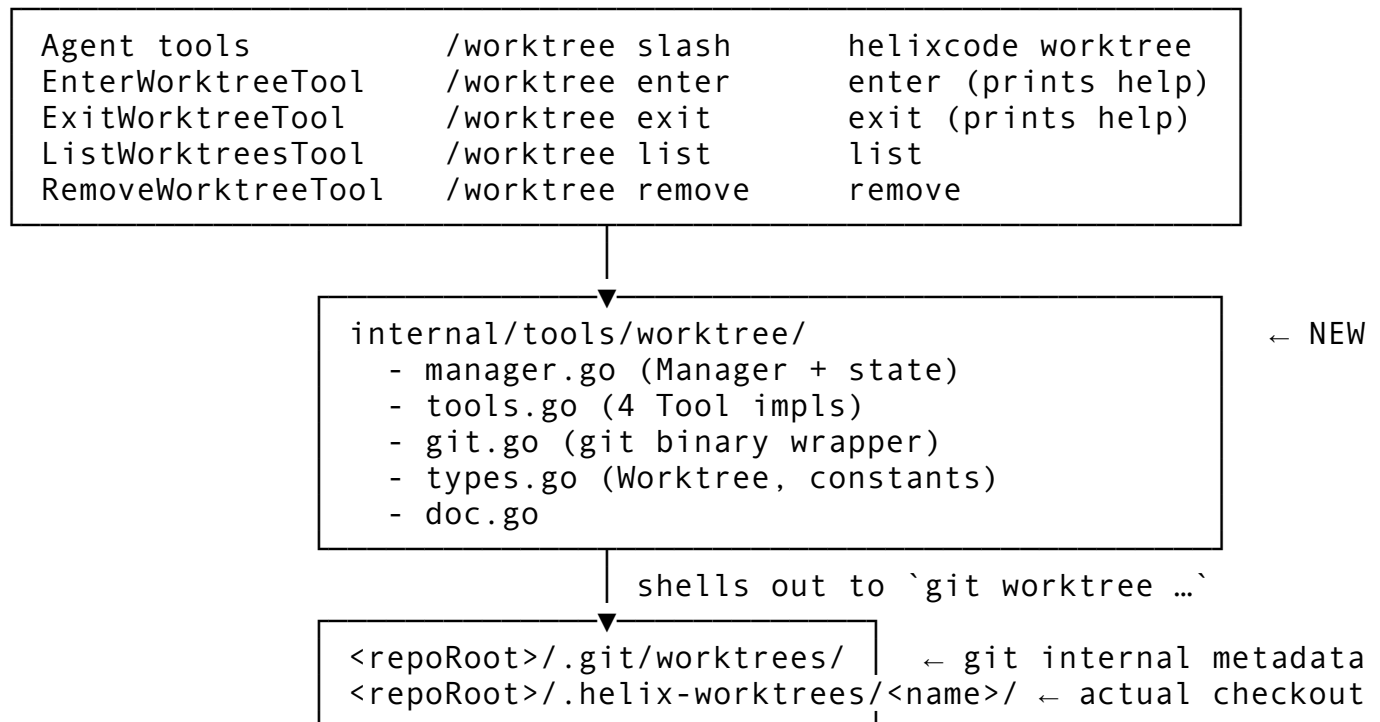
1.4 Success criteria

- **S1.** `make verify-compile` exits 0 with the new package + provider wiring + CLI subcommand.
- **S2.** Unit tests for `internal/tools/worktree/` pass with `-race`. Most use `REAL git` against `t.TempDir()` ephemeral repos (shelling out is fast enough; no value in mocking).
- **S3.** Integration test (`-tags=integration`, no mocks) demonstrates: a real `EnterWorktree` against an ephemeral repo creates `.helix-worktrees/feature-x/` with the same `README.md` content as `main`; a commit on `feature-x` does not change `main`'s HEAD; `RemoveWorktree` cleans the directory and the branch metadata.
- **S4.** Challenge under `tests/e2e/challenges/worktree/` runs three scenarios with runtime evidence pasted into the close-out commit body. The mutation-test recipe in the README ensures the Challenge would fail if `ValidateName`'s regex check were disabled.

- **S5.** Anti-bluff smoke (`grep -rn "simulated\|for now\|TODO implement\|placeholder" internal/tools/worktree/`) returns zero hits.
- **S6.** All 4 agent tools registered with `tools.ToolRegistry.helixcode worktree list` smoke-tested. `/worktree slash` command discoverable via `registry.Get("worktree")`.
- **S7.** Per-session state: `session.Manager.GetCurrentWorktree()` returns the path set by `EnterWorktree` and is reset by `ExitWorktree`.

2. Architecture

2.1 Topology



A single `*worktree.Manager` instance is constructed at CLI startup with `repoRoot = git rev-parse --show-toplevel` (fallback to `os.Getwd()`). The same instance is shared by:

- Tool registration (passed to each `*Tool` constructor).
- The slash command (looked up via the existing `internal/commands/builtin` registry).
- The Cobra subcommand group (passed in from `main.go`).
- The session manager (used to keep `currentWorktree` consistent).

2.2 Why this shape

- **One new sub-package.** Same pattern as F02 (`permissions/`) and F03 (`persistence/`). Consistent file layout makes the codebase's tools layer easy to navigate.
- **Shells out to git.** Existing `internal/tools/git/` already does this (8 `exec.Command` callsites). Consistent dependency surface; no new library.
- **Per-session state in one field, not a parallel file.** The porting doc's `internal/session/worktree_state.go` would add a new file for state that's already naturally a property of the session. A 5-line field add to `session.Manager` is cleaner.

- `internal/tools/git/ untouched`. That package is focused on auto-commit + attribution + message generation. Worktree management is a different concern — a workflow operation, not a commit-metadata one. Mixing them dilutes both.

2.3 Component responsibilities

Component	Responsibility
<code>types.go</code>	Worktree struct (Name, Path, Branch); constants (WorktreeNameReg, WorktreeNameMaxLength, WorktreeDir); compiled <code>worktreeNamePattern *regexp.Regexp</code>
<code>git.go</code>	Pure functions wrapping <code>exec.Command("git", ...)</code> : <code>gitWorktreeAdd(ctx, repoRoot, branch, path) (string, error)</code> , <code>gitWorktreeAddNewBranch(ctx, repoRoot, branch, path) (string, error)</code> , <code>gitWorktreeRemove(ctx, repoRoot, path, force bool) (string, error)</code> , <code>gitWorktreeList(repoRoot) (string, error)</code> , <code>gitStatusPorcelain(ctx, (string, error)</code> , <code>gitRevParseToplevel(ctx, cwd) (string, error)</code> . Each returns (combinedOutput, err). Separated for unit-clarity.
<code>manager.go</code>	Manager struct (repoRoot, currentWorktree, mu); <code>NewManager(repoRoot)</code> , <code>ValidateName(name)</code> , <code>EnterWorktree(ctx, name, baseBranch) (string, error)</code> , <code>ExitWorktree()</code> , <code>ListWorktrees(ctx) ([]Worktree, error)</code> , <code>RemoveWorktree(ctx, name) error</code> , <code>GetCurrentDirectory() string</code> , <code>IsIsolated() bool</code>
<code>tools.go</code>	4 <code>tools.Tool</code> interface impls. Each holds a <code>*Manager</code> reference; <code>Execute(ctx, params)</code> adapts to Manager calls; <code>Schema()</code> returns <code>tools.ToolSchema</code> ; <code>Validate(params)</code> checks parameter types/n
<code>cmd/cli/worktree_cmd.go</code>	Cobra helixcode <code>worktree {list,enter,exit,remove}</code> group <code>enter/exit</code> print help “use from inside helixcode chat session” and 1. <code>list/remove</code> call Manager directly (since they’re stateless). The CL main dispatcher (added in F02) intercepts <code>os.Args[1] == "worktree"</code> before <code>flag.Parse()</code> .
<code>internal/commands/worktree_command.go</code>	WorktreeCommand struct implementing <code>commands.Command</code> (Name, Aliases, Description, Usage, Execute). Execute dispatches on <code>cmdCtx.Args[0]: empty/list → Manager.ListWorktrees; enter → EnterWorktree; exit → ExitWorktree; remove → RemoveWorktree</code> .
<code>internal/commands/builtin/register.go</code> (extended)	Add <code>registry.Register(commands.NewWorktreeCommand(manager))</code> alongside the existing <code>NewPermissionsCommand</code> registration. Update <code>GetBuiltinCommandNames</code> and <code>GetBuiltinCommandAliases</code> .
<code>internal/session/manager.go</code> (extended)	Add <code>currentWorktree string</code> field + <code>GetCurrentWorktree() string</code> + <code>SetCurrentWorktree(s string)</code> methods. Use the session’s existing mutex (or add <code>sync.RWMutex</code> if absent).
<code>cmd/cli/main.go</code> (extended)	Construct <code>*worktree.Manager</code> at startup with <code>repoRoot</code> resolved via <code>gitRevParseToplevel</code> (fallback to <code>os.Getwd()</code>). Inject into tool registry, Cobra subcommand group, and <code>WorktreeCommand</code> constructor.

Component	Responsibility
<code>.gitignore</code> (<code>root + helix_code/.gitignore</code>)	Add <code>.helix-worktrees/</code> entry to both.

3. Data shapes

3.1 Constants

```
// internal/tools/worktree/types.go
package worktree

import "regexp"

const (
    // WorktreeNameRegex constrains worktree names to
    // alphanumerics + . _ - .
    // Matches claude-code's own validation pattern.
    WorktreeNameRegex = `^[a-zA-Z0-9._-]+$`
    WorktreeNameMaxLength = 64
    WorktreeDir          = ".helix-worktrees"
)

var worktreeNamePattern = regexp.MustCompile(WorktreeNameRegex)
```

3.2 Worktree

```
type Worktree struct {
    Name    string `json:"name"`           // user-facing
           identifier
    Path    string `json:"path"`           // absolute path on
           disk
    Branch  string `json:"branch,omitempty"` // best-effort, may
           be empty
}
```

3.3 Manager

```
type Manager struct {
    repoRoot      string
    currentWorktree string // "" = main worktree
    mu             sync.RWMutex
}

func NewManager(repoRoot string) *Manager
func (m *Manager) ValidateName(name string) error
func (m *Manager) EnterWorktree(ctx context.Context, name,
    baseBranch string) (string, error)
```

```

func (m *Manager) ExitWorktree()
func (m *Manager) ListWorktrees(ctx context.Context)
    ([]Worktree, error)
func (m *Manager) RemoveWorktree(ctx context.Context, name
    string) error
func (m *Manager) GetCurrentDirectory() string
func (m *Manager) IsIsolated() bool

```

GetCurrentDirectory returns m.currentWorktree if non-empty, else m.repoRoot. IsIsolated returns m.currentWorktree != "". Both use RLock. EnterWorktree / ExitWorktree / RemoveWorktree use Lock.

4. Branch-creation semantics

```

EnterWorktree(name, baseBranch):
    if err := ValidateName(name); err != nil:
        return "", err

    branch := baseBranch
    if branch == "":
        branch = name

    path := filepath.Join(repoRoot, WorktreeDir, name)

    if path exists on disk:
        # Existing worktree – validate clean, reuse
        out := gitStatusPorcelain(ctx, path)
        if out != "":
            return "", "worktree %q has uncommitted changes – clean or rem
        currentWorktree = path
        return path, nil

    if err := mkdir <repoRoot>/.helix-worktrees/:
        return "", err

    # Try existing branch first
    out, err := gitWorktreeAdd(ctx, repoRoot, branch, path)
    if err != nil:
        # Fall back to creating a new branch
        out2, err2 := gitWorktreeAddNewBranch(ctx, repoRoot, branch, path)
        if err2 != nil:
            return "", composite-error wrapping both outputs

    currentWorktree = path
    return path, nil

```

The two-step “try existing, fall back to new” pattern matches the porting doc.

5. Submodule handling

Per §1.2 G4: EnterWorktree does NOT initialise submodules. The Description returned by EnterWorktreeTool.Description() documents this:

“Enter a named git worktree for isolated development. Creates the worktree if it doesn’t exist (using the worktree name as the branch name when no base-branch is supplied). Submodules are NOT initialised — the meta-repo and the inner Go module at helix_code/ are present, but submodule directories under helix_agent/, dependencies/, etc. are empty placeholders. If your work needs submodule code, run git submodule update --init --recursive from inside the worktree using Bash.”

This becomes part of the LLM-visible tool documentation.

6. CLI surface

6.1 Cobra subcommands

```
helixcode worktree list           # via Manager.ListWorktree
helixcode worktree remove <name> # via Manager.RemoveWorktree
helixcode worktree enter <name> [base-branch] # prints help + exits 1 (
helixcode worktree exit           # same
```

The enter/exit subcommands DELIBERATELY exit non-zero with an explanatory message because session-scoped state can’t persist across stateless CLI invocations. The agent inside an interactive helixcode chat session uses the tool or slash command instead.

6.2 Agent tools

Registered with tools.ToolRegistry at startup. Each accepts the *Manager via constructor injection.

Tool	Parameters	Returns
EnterWorktree	name string (required), baseBranch string (optional)	{path: string}
ExitWorktree	none	{exited: bool}
ListWorktrees	none	{worktrees: []Worktree}
RemoveWorktree	name string (required)	{removed: bool}

Each tool’s Schema() returns a ToolSchema with Type: "object", Properties enumerating the parameters (with description strings for the LLM), and Required listing the mandatory ones.

6.3 Slash command

```
/worktree                                # show current state + list (default)
/worktree list                           # explicit list
/worktree enter <name> [branch]          # mutates manager + session state
/worktree exit
/worktree remove <name>
```

Implementation pattern: same as F02's /permissions slash command — a single struct implementing commands. Command, registered in `internal/commands/builtin/register.go`.

7. Per-session state

`internal/session/manager.go` gains:

```
type Manager struct {
    // existing fields ...
    currentWorktree string

    // mu sync.RWMutex (use the session's existing mutex if
    // present;
    // add one if not — depends on the current state of session/
    // manager.go)
}

func (m *Manager) GetCurrentWorktree() string
func (m *Manager) SetCurrentWorktree(s string)
```

Plumbing flow: `EnterWorktreeTool.Execute` calls BOTH `worktree.Manager.EnterWorktree(...)` (in-process state) AND `session.Manager.SetCurrentWorktree(path)` (session-persistent state). On session resume (P1-F11 future), the session's `currentWorktree` is read and propagated back to the `WorktreeManager`. F04 wires the field; the resume path is N2-out-of-scope.

For F04, the field's only consumer outside the worktree wiring is the slash command's /worktree (no-args) display, which reports current state by reading the session's `GetCurrentWorktree()`.

8. Error handling

Case	Behaviour
Empty name	<code>ValidateName</code> returns "worktree name cannot be empty"
Name >64 chars	<code>ValidateName</code> returns "worktree name exceeds 64 characters"
Name fails regex	

Case	Behaviour
	<code>ValidateName</code> returns "worktree name %q does not match pattern <code>^[a-zA-Z0-9._-]+\$</code> "
<code>repoRoot</code> is not a git repo	<code>gitWorktreeAdd</code> fails; error wraps git's verbatim output
Pre-existing worktree dir is dirty	<code>EnterWorktree</code> returns "worktree %q has uncommitted changes – clean or remove first"
Pre-existing worktree dir is clean	reuse; update <code>currentWorktree</code> ; no git commands run
<code>EnterWorktree</code> while already in another worktree	log WARN; replace <code>currentWorktree</code> ; previous worktree's checkout untouched on disk
<code>RemoveWorktree</code> on the worktree we're inside	error: "cannot remove the current worktree; <code>ExitWorktree</code> first"
<code>RemoveWorktree</code> fails	retry with <code>-f</code> ; if both fail, return composite error wrapping both outputs
<code>ListWorktrees</code> when <code>.helix-worktrees/</code> doesn't exist	return empty slice + nil error
Branch already checked out by another worktree	git refuses; error wraps verbatim output ("already used by another worktree")
Concurrent <code>EnterWorktree</code> calls (same name)	<code>Manager.mu.Lock</code> serialises; both calls resolve to the same path

Error messages are passed verbatim through to the LLM so it can react meaningfully (e.g., upon "branch already checked out", the agent can call `ListWorktrees` to inspect).

9. Testing strategy (CONST-035 / Article XI §11.9)

9.1 Unit (internal/tools/worktree/*_test.go)

- `ValidateName`: empty, 65-char, valid (feature-x, _pre-release, v1.2.3-rc1), invalid (/, . . , spaces, unicode, control chars).
- `git.go` wrappers: each tested against a real ephemeral repo via `t.TempDir()` + `git init` + `git commit`. Asserts `gitWorktreeAdd` produces a checkout, `gitStatusPorcelain` empty for clean repo, etc.
- `Manager.EnterWorktree`:
 - Existing branch path: ephemeral repo with feature-x already committed; `EnterWorktree("feature-x", "")` succeeds; verify via `git worktree list`.
 - New branch path: ephemeral repo without feature-x; `EnterWorktree("feature-x", "")` falls through to `-b feature-x` and succeeds.
 - Dirty pre-existing worktree: enter once, write a file, enter again → returns the dirty error.

- `repoRoot` not a git repo: `t.TempDir()` directly (no `git init`) → returns the verbatim git error.
- `Manager.ListWorktrees`: empty dir, multiple entries, file-in-`.helix-worktrees/` ignored (only directories counted).
- `Manager.RemoveWorktree`: removes both the dir and the branch metadata; refuses to remove the current worktree.
- `Manager.GetCurrentDirectory / IsIsolated`: defaults (returns `repoRoot`, false), after enter (returns worktree path, true), after exit (back to defaults).

Mocks ALLOWED at this layer per CLAUDE.md, but most tests use REAL git.

9.2 Integration (tests/integration/worktree/worktree_integration_test.go, -tags=integration)

- Uses a real `t.TempDir()` + `git init` repo with a real seed commit on main.
- Asserts: `EnterWorktree("feature-x", "")` creates `.helix-worktrees/feature-x/`; `git -C <path> branch --show-current` returns `feature-x`; the worktree contains the same `README.md` content as main.
- Asserts: writing a file in the worktree, committing on `feature-x`, then verifying main's HEAD is unchanged.
- Asserts: `RemoveWorktree("feature-x")` deletes the dir and the branch metadata (verifies via `git worktree list`).

No mocks.

9.3 Challenge (tests/e2e/challenges/worktree/)

End-to-end via a Go-built driver (mirrors F03's pattern):

- **S1** — Create worktree, write a file, commit on the new branch, verify main's HEAD is unchanged. (Proves real isolation.)
- **S2** — Re-enter the same worktree (already exists, clean) → no error, returns same path. (Proves idempotent re-entry.)
- **S3** — Try to enter a worktree whose name fails validation: `../etc`, empty (`""`), 65-char string. All three rejected with the appropriate error. (Proves the security guard.)

Runtime evidence: per-scenario stdout + filesystem inspection (`ls -la .helix-worktrees/` + `git worktree list` output) pasted into the close-out commit body.

9.4 Anti-bluff smoke

```
grep -rn "simulated\\|for now\\|TODO implement\\|placeholder"
internal/tools/worktree/ must be empty.
```

9.5 Mutation test (CONST-039)

Temporarily comment out the regex check in `ValidateName`. Re-run Challenge — S3 MUST FAIL because `../etc` is now accepted. Revert and confirm PASS.

10. Sub-task plan

13 tasks (matches F02's surface area):

#	Task	Outputs
T01	Bootstrap 06_phase_1_evidence.md \$F04 + advance PROGRESS to F04-active + add .helix- worktrees/ to root + helix_code/.gitignore	docs + gitignore only
T02	internal/tools/ worktree/types.go + doc.go skeleton (constants + Worktree struct)	compile-only
T03	git.go thin git-binary wrappers + tests against real ephemeral repo (TDD)	unit tests pass
T04	manager.go: Manager struct + ValidateName + GetCurrentDirectory + IsIsolated (TDD)	unit tests pass
T05	Manager.EnterWorktree (TDD; existing-branch path, new-branch path, dirty rejection)	unit tests pass
T06	Manager.ExitWorktree + Manager.ListWorktrees + Manager.RemoveWorktree (TDD)	unit tests pass
T07	tools.go: 4 tools.Tool interface impls + tests	unit tests pass
T08	internal/session/ manager.go: add currentWorktree field + getter/setter (TDD)	unit tests pass
T09	cmd/cli/ worktree_cmd.go Cobra subcommand group + dispatcher addition + tests	unit tests pass; helixcode worktree list smoke- tested
T10	internal/commands/ worktree_command.go slash command + register in builtin/register.go (TDD)	unit + builtin-registry tests
T11	cmd/cli/main.go startup wiring: construct worktree.Manager;	-tags=integration test passes

#	Task	Outputs
	integration test under <code>tests/integration/worktree/</code> (no mocks, real ephemeral repo)	
T12	Challenge under <code>tests/e2e/challenges/worktree/</code> with three scenarios from §9.3	Challenge PASS in commit
T13	Feature 4 close-out: anti-bluff scan, <code>make verify-foundation</code> , push to all four remotes (no force)	PROGRESS flipped to F05

13 sub-commits expected. F05 (Hook-Based Extensibility) unblocked when T13 lands.

11. Out of scope for F04

Listed in §1.3 (N1–N7). Summarised here: subagent isolation (P1-F15), session resume (P1-F11), submodule auto-init, worktree-aware Read/Write/Bash tools (later wiring), F02/F03 follow-ups, `go-git`, out-of-repo storage.

12. Risks and mitigations

Risk	Mitigation
HelixCode is itself sometimes a submodule of another repo; <code>.helix-worktrees/</code> inside a submodule may confuse <code>git worktree</code>	<code>Manager.NewManager</code> uses <code>gitRevParseToplevel</code> to resolve the actual repo root; T11 integration test creates an ephemeral repo and verifies the path layout.
Branch already checked out elsewhere	Plan documents the error case; tool surfaces <code>git</code> 's verbatim output. The agent can call <code>ListWorktrees</code> to investigate.
Concurrent <code>EnterWorktree</code> calls (race on same name)	<code>Manager.mu</code> serialises mutations; concurrent enters with the same name converge on the same path.
<code>RemoveWorktree</code> on a worktree that was manually deleted	<code>git worktree remove</code> may fail; falls back to <code>-f</code> ; documented.
<code>.helix-worktrees/</code> name collides with user data	Unlikely (dot-prefix + specific name). Collision triggers a startup-time error or a documented <code>git worktree add</code> failure.
HelixCode meta-repo's tracked-subdirectory-not-submodule layout for <code>helix_code/</code> makes <code>git worktree</code> semantics unusual	The inner Go module IS present in the worktree (it's a tracked subdirectory). T11 integration test verifies.
<code>cmd/cli/main.go</code> is large and tangled (per F02/F03 review notes)	T09 + T11 only ADD a few lines. No restructuring.

Risk	Mitigation
F02's permissions wiring gap + F03's persistence wiring gap also touch <code>cmd/cli/main.go</code>	F04's wiring is independent (different fields, different methods). The three features coexist on the CLI struct.

13. References

- Synthesis spec: `docs/superpowers/specs/2026-05-04-cli-agent-fusion-synthesis-design.md` §4.1 (Phase 1 charter)
- Porting doc: `docs/improvements/04_main_plan_step_02/kimi_agent_helix_cli_integration_blueprint/porting_claud_code.md` §Feature 4
- Predecessor spec: `docs/superpowers/specs/2026-05-05-p1-f03-tool-result-persistence-design.md` (commit `f813fc9`)
- Predecessor plan: `docs/superpowers/plans/2026-05-05-p1-f03-tool-result-persistence.md` (commit `d33f674`)
- Evidence file (live): `docs/improvements/06_phase_1_evidence.md`
- Existing infrastructure being audited (NOT modified except for wiring):
 - `helix_code/internal/tools/git/` — auto-commit + attribution + message generation; uses `os/exec` to shell out to git (8 callsites). Pattern reused; package itself not modified.
 - `helix_code/internal/tools/registry.go` — Tool interface (`Name()`, `Description()`, `Execute(ctx, params) (interface{}, error)`, `Schema() ToolSchema`, `Category() ToolCategory`, `Validate(params) error`); `CategoryShell` is the closest existing category. F04 introduces no new category — worktree tools use `CategoryShell` (closest semantic match).
 - `helix_code/internal/commands/command.go` — slash command Command interface
 - `helix_code/internal/commands/builtin/register.go` — slash-command registry entry point
 - `helix_code/internal/session/manager.go` — session state (extended with one field)
- Constitutional anchors:
 - Article XI §11.9 — Anti-Bluff Forensic Anchor
 - CONST-035 — Zero-Bluff Mandate
 - CONST-039 — Challenge System Integrity (mutation testing mandatory)
 - CONST-042 — No-Secret-Leak (N/A; worktree paths may contain user data but not credentials)
 - CONST-043 — No-Force-Push (close-out commit T13 pushes without force)

End of P1-F04 Git Worktree Agent Isolation design spec.